SREC Sponsorship CRM: Design Summary

Nov 8, 2026 · @AJ

What it is

An internal CRM for the NYU Stern Real Estate Club sponsorship team: a Google Sheet is the database and a Google Apps Script web app (plain HTML, CSS and JavaScript, no framework, no build step) is the UI on top. Access is controlled entirely by sheet sharing; teammates sign in with their NYU Google account and nobody needs a Claude login.

Built in October 2026. Claude (claude.ai) designed the system, built the sheet, wrote the first version of the code and produced the data imports; Claude Code implemented later iterations from one prompt per feature, pushed with `clasp`. This document is the handoff for reusing the pattern in a new project.

Architecture and why

The sheet holds the data, a bound Apps Script serves the app, and the app runs as whoever opened it. Each choice below was made to keep the team on tools they already use and to leave nothing to maintain.

- **Google Sheet as source of truth.** The team already lives in Sheets, can bulk-paste data, and needs no new login. Core fields are edited in the sheet; the app is the reading and logging layer.
- **Apps Script bound to the sheet** (Extensions → Apps Script). The script lives with the sheet, is owned by the same account, and transfers with it when the board turns over.
- **Web app deployed with `executeAs: USER_ACCESSING` and `access: DOMAIN`.** The script reads the sheet as the signed-in user, so anyone not shared on the sheet gets an access error; remove them from the sheet and they are out of the app. Domain access (NYU) avoids the unverified-app warning for internal users. Rule recorded in CLAUDE.md: never switch to `USER_DEPLOYING`, which would let any link-holder read as the owner.
- **Dev workflow.** `clasp login`, a hand-written `.clasp.json` with the script ID and `rootDir: "."`, `clasp push --force`, test on the `/dev` URL (Deploy → Test deployments, always the latest push), then `clasp deploy -i <deploymentId>` to update production without changing the URL. Claude Code runs this from the project folder. A `CLAUDE.md` holds the data model, server API and no-go rules so each iteration

starts with context. Steps inside Google's UI (upload, share, authorize, the `clasp login` browser sign-in) stay with the human; everything terminal-side goes to Claude Code.

Alternative considered	Why it lost
Claude artifact with the Drive connector	Teammates would need a Claude login and their own Drive connection; no public sharing
Static site + Google Sheets API with OAuth	Needs a Cloud project, a test-user list, weekly token expiry in testing mode
Sheet published to the web as CSV	Makes real contact data public to anyone with the link
AppSheet	Not HTML; access managed in AppSheet rather than by sheet sharing

Data model

One tab per object; the contact is the main object, the organization sits above it, and events, attendance, panels, outreach and commitments hang off those two.

Tab	Key	Holds
Contacts	`Contact_ID` (C001…)	Name, Title, Organization (dropdown → Organizations), Email, Phone, LinkedIn, Stern_Alum, Stern_Program, Class_Year, Source, Owner, Status, Tags, Notes, Approval, Warm_Intro, Ask_For
Organizations	`Org_Name`	Type, Sponsor_Status, Current_Tier, Website, HQ_City, Primary_Contact_ID, Owner, Notes, Priority, Outreach_Stage, Next_Step, Next_Step_Date, Sponsorship_Target, Ask_For, Warm_Intro, Approval

Tab	Key	Holds
Events	`Event_Name`	Event_Type, Date, Location, Semester, Academic_Year, Sponsor_Orgs, Notes
Event_Attendance	Contact_ID + Event_Name	Contact (formula), Role, Registered, Attended, Notes
Outreach_Log	Contact_ID	Date, Contact (formula), Channel, Owner, Purpose, Summary, Outcome, Next_Step, Next_Step_Date
Panelists	Contact_ID + Event_Name	Contact (formula), Panel_Title, Role, Topic, Notes
Sponsorship_Commitments	Org_Name + Academic_Year	Semester, Tier, Amount, Status (Pitched → Verbal → Committed → Invoiced → Paid), Contact_ID, Date_Committed, Benefits_Used, Owner
Lists	one list per column	Every dropdown's values; data validation on all tabs points here
Templates	`Template_Name`	Channel, Purpose, Subject, Body with merge fields
Alumni_Reference	none	1,442 verified rows from a 2016 alumni database, lookup only; the app ignores it

Rules the design depends on:

- Keys are human-readable text (firm name, event name), not hidden IDs, except `Contact_ID` because names collide. Child tabs use dropdowns fed from the parent tab's key column, so joins stay exact without IDs the team has to memorize.
- Child tabs carry a `Contact` column holding a VLOOKUP formula so people entering rows see a name, not an ID. Formulas are pre-filled for about 100 rows; the server writes the same formula when it appends a row.
- Sponsorship money and status live on the organization, not the person.

- Column order does not matter to the app; header names do. New columns go to the right. This is what let assignment, approval and warm-intro columns be added weeks later without touching the read code.
- Row 1 is always the header; a row with a blank key is ignored; a `Lists_additions` tab in each import tells the user which dropdown values to add so validation accepts new data.

Server design (Code.gs)

One read function returns every tab as strings; one write helper appends by header name. Everything else is validation and plumbing.

- `getData()` reads every tab in one pass with `getDisplayValues()` and returns arrays of row objects keyed by header, plus the Lists, the signed-in user's email and the sheet URL. Display values keep dates as `yyyy-mm-dd` strings and currency as the sheet shows it, and sidestep the fact that `google.script.run` cannot return Date objects. All joins happen in the client.
- `appendRecord_(tab, record)` is the only write path: find the first row whose key cell is blank (not `appendRow`, because formula-filled rows count as content), map the record onto headers by name, convert date columns to real Dates and Amount to a Number, and write the lookup formula into the Contact column. `addOutreach` and `addAttendance` check that the referenced contact or event exists before writing.
- `openSpreadsheet_` throws `NO_ACCESS` when the user cannot open the sheet, so the client shows a plain "ask to be shared on the sheet" screen instead of a stack trace.
- Password-gated email: `getData` strips the Email value and returns `Has_Email` instead; `revealEmail(contactId, password)` compares against a value in Script Properties (set once with `setEmailPassword`) and returns the address. Stated limitation: anyone shared on the sheet can still see the Email column there, so this is a courtesy gate in the app, not security.
- `updateOrgOutreach(orgName, fields)` is the one edit-in-place exception, writing only Owner, Priority, Outreach_Stage, Next_Step and Next_Step_Date on the firm's row. Everything else is edited in the sheet by design.
- Manual test: `test_getData()` run from the editor prints row counts to the Execution log; the first run triggers the OAuth consent for the spreadsheets and email scopes.

Frontend design (index.html)

A single file, one state object, three render functions, and a master-detail layout built for scanning rather than decoration.

- **State and rendering.** One `state` object; `render()` draws the shell, `rerenderList()` and `rerenderDetail()` redraw a single column. Every user action mutates state and calls one of those. All lookups (by ID, by firm, by event, grouped children) are built once in `buildIndex()` after each load. Every value passes through `esc()` because sheet content is user-typed.
- **Layout.** Left rail (Contacts, Organizations, Outreach/Targets, Events), a 400px list column with search and filters, and a detail panel. Desktop-first; below 980px the three columns stack.
- **Contact detail.** Name, title and firm; chips for approval, alum status, source and tags; then a four-cell history strip: events attended, panels spoken on, last touch (date, channel, by whom), and the firm's sponsor status and tier. Below that the outreach timeline (newest first), events table, panels table, the firm's commitments, and raw details with a pointer to the sheet row for edits.
- **Organization detail.** People at the firm, commitments by year, recent outreach across all its contacts, total committed or paid. **Event detail.** Panels grouped by title, attendees, firms represented, and an Add attendee form.
- **Write-back forms** are inline panels inside the detail column, not modals. On save they reload data and show a toast; on failure they stay open with the error.
- **Visual language.** One accent (NYU violet `#57068C`), IBM Plex Sans with a system fallback, status carried by tinted chips (green, amber, red, grey, violet mapped from list values), light borders instead of fills, no illustration. It reads as a working tool.
- Cross-links use `data-go="<view>" data-key="<key>"` so any name anywhere in the app opens its record.

Outreach and assignment features

Assignment happens at the firm, approval gates the button, and every send from the app logs itself.

- **Owner vs Warm_Intro.** `Owner` is the sponsorship team member assigned to work the relationship, set on the organization and inherited by its contacts unless overridden. `Warm_Intro` is the board member who already knows the person (the source list's

"Who's Contact"). Keeping them separate was deliberate: one is a work assignment, the other is a relationship fact.

- **Approval** means cleared to contact: `Good to go`, `Good to go (tell Jess first)`, `Hold`, `Not reviewed`, or blank (blank = no restriction known, e.g. past speakers). The Do outreach button is hidden for Hold and Not reviewed with a one-line reason, and shows a heads-up banner for the tell-Jess case.
- **Ask_For** lists which asks apply to a person or firm (Conference, Deal Lab, Case competition mentor, Academic & Industry Relations, Sponsorship) as chips. `Sponsorship_Target` on firms is Yes / Maybe / No.
- **Pipeline on Organizations.** `Priority` A/B/C, `Outreach_Stage` (Not started → Researching → Reached out → In conversation → Pitched → Committed → Declined → Parked), `Next_Step`, `Next_Step_Date`, editable from the firm's detail panel. `Sponsor_Status` stays separate as relationship history.
- **Outreach / Targets view.** Firms grouped by Owner with Unassigned last, showing priority, stage, approval, warm intro, named contacts, last touch and next step; filters by owner, stage, approval and target; a toggle hides Parked and Declined.
- **Do outreach flow.** Pick a channel (LinkedIn default, Email), pick a template from the Templates tab, see a preview with `{First_Name} {Organization} {Owner} {Event}` filled in, then act. Email builds a `mailto:` link with subject and body (kept under about 1,800 characters) after the password unlock. LinkedIn copies the message to the clipboard and opens the profile, because LinkedIn has no compose deep-link; with no profile URL it opens a people search. Either click logs an Outreach_Log row ("Sent via app: …", Outcome `No response yet`, Owner = signed-in user) and refreshes. Templates live in the sheet so the team edits them without code.
- **Contact header** shows "Assigned to" and "Last updated" (most recent outreach date) so the list doubles as a who-owns-whom and when-did-we-last-talk view.

Data import pipeline

A Python script turned five messy sources into paste-ready tabs with stable IDs and a review list, and never guessed what it could not know.

Source	What it contributed
Two conference slide decks (Luxury Summit 11/7/25, POWER 3/6/26)	Every named speaker as a contact, a Panelists row per slot, attendance marked

Source	What it contributed
25-26 speaker wishlist	Contacts with Source "Speaker wishlist", approvals and sourcing board member in Notes
Master calendar 25-26 and 26-27	69 events; named speakers linked as Panelists; attendance only for past events
2016 alumni database (4,200 rows)	Former board members promoted to Contacts; 1,442 verified rows to Alumni_Reference; name matches set Stern_Alum = Yes
Sept 2026 speaker and sponsor outreach list	73 new contacts, 82 new firms, and the Approval, Warm_Intro, Ask_For and Sponsorship_Target columns

How it works:

- Every tab is written with the live sheet's exact headers in the user's column order, so each tab is a select-copy-paste into Google Sheets. Sheets-to-Sheets copy keeps dates and the Contact formulas; the import carries the formula in the Contact column so pasting does not blank it.
- Contacts are deduped on a normalized first+last key with an override map for spelling variants, nicknames and middle names. Firms pass through an alias map (one canonical name per firm); Type is inferred only for well-known firms.
- IDs are deterministic and stable across versions: sources load in a fixed order and later sources merge into existing entries, so the v2 import kept every v1 ID and appended new people from C225. A newer source may overwrite a firm from an older one, except when the older one is a conference deck.
- Unknowable fields stay blank: Sponsor_Status, Owner, Status, a firm the slide omitted. Anything inferred or merged by name match gets a row in a **Review** tab for the human. Alumni name matches set the flag but are listed for confirmation.
- The 2016 database went to a reference tab rather than Contacts; 4,200 stale rows would have buried the working list.
- Lists_additions tells the user which dropdown values to add before pasting.

Lessons and pitfalls

Most of the friction was in setup, not design; the patterns below are the ones worth carrying into the next project.

What worked

- A numbered checklist the user ticks off, with questions as they arise, beat long explanations for a multi-tool setup.
- One Claude Code prompt per feature, naming exact column names, list values and acceptance behavior, and ending with "push and tell me to test on the /dev URL".
- CLAUDE.md as the contract between sessions: data model, server API, conventions, and a short list of things not to do.
- Reading by header name, not position, so the sheet could grow columns without code changes.
- Flagging instead of guessing in imports: a Review tab the human works through keeps bad data out and keeps trust in the tool.

Pitfalls hit

- Uploading a 20KB+ workbook through a chat tool as base64 is fragile; build the file and let the user upload or import it.
- Fresh Mac: Node and npm were not installed (`brew install node`); zsh chokes on pasted `<PLACEHOLDER>` brackets; Claude Code's own login expires (`/login`); Claude Code asks permission per command, which is normal.
- Do not `clasp clone` into a folder that already holds the real files; it pulls Google's placeholders over them. Write `.clasp.json` by hand and `clasp push --force` .
- Apps Script specifics: `google.script.run` cannot return Date objects (use display values); `appendRow` lands after pre-filled formula rows (find the first blank key cell instead); `setValue` with a date string stores text (convert to Date server-side); users get a Google authorization prompt on first run.
- Ownership cannot move from a consumer Gmail into a Workspace domain; re-import the sheet in the target account instead.
- A password check inside an app that runs as the viewer cannot hide data the viewer can already see in the sheet; say so rather than imply security.